12.1. Introduction

Full Text Searching (or just text search) provides the capability to identify natural-language documents that satisfy a query , and optionally to sort them by relevance to the query. The most common type of search is to find all documents containing given query terms and return them in order of their similarity to the query. Notions of query and similarity are very flexible and depend on the specific application. The simplest search considers query as a set of words and similarity as the frequency of query words in the document.

Textual search operators have existed in databases for years. PostgreSQL has ~ , ~* , LIKE , and ILIKE operators for textual data types, but they lack many essential properties required by modern information systems:

• There is no linguistic support, even for English. Regular expressions are not sufficient because they cannot easily handle derived words, e.g., satisfies and satisfy . You might miss documents that contain satisfies , although you probably would like to find them when searching for satisfy . It is possible to use OR to search for multiple derived forms, but this is tedious and error-prone (some words can have several thousand derivatives).

• They provide no ordering (ranking) of search results, which makes them ineffective when thousands of matching documents are found.

• They tend to be slow because there is no index support, so they must process all documents for every search.

Full text indexing allows documents to be preprocessed and an index saved for later rapid searching. Preprocessing includes:

• Parsing documents into tokens . It is useful to identify various classes of tokens, e.g., numbers, words, complex words, email addresses, so that they can be processed differently. In principle token classes depend on the specific application, but for most purposes it is adequate to use a predefined set of classes. PostgreSQL uses a parser to perform this step. A standard parser is provided, and custom parsers can be created for specific needs.

• Converting tokens into lexemes . A lexeme is a string, just like a token, but it has been normalized so that different forms of the same word are made alike. For example, normalization almost always includes folding upper-case letters to lower-case, and often involves removal of suffixes (such as s or es in English). This allows searches to find variant forms of the same word, without tediously entering all the possible variants. Also, this step typically eliminates stop words , which are words that are so common that they are useless for searching. (In short, then, tokens are raw fragments of the document text, while lexemes are words that are believed useful for indexing and searching.) PostgreSQL uses dictionaries to perform this step. Various standard dictionaries are provided, and custom ones can be created for specific needs.

• Storing preprocessed documents optimized for searching . For example, each document can be represented as a sorted array of normalized lexemes. Along with the lexemes it is often desirable to store positional information to use for proximity ranking , so that a document that contains a more “ dense ” region of query words is assigned a higher rank than one with scattered query words.

Dictionaries allow fine-grained control over how tokens are normalized. With appropriate dictionaries, you can:

• Define stop words that should not be indexed.

• Map synonyms to a single word using Ispell .

• Map phrases to a single word using a thesaurus.

• Map different variations of a word to a canonical form using an Ispell dictionary.

• Map different variations of a word to a canonical form using Snowball stemmer rules.

A data type tsvector is provided for storing preprocessed documents, along with a type tsquery for representing processed queries (Section 8.11). There are many functions and operators available for these data types (Section 9.13), the most important of which is the match operator @@ , which we introduce in Section 12.1.2 . Full text searches can be accelerated using indexes (Section 12.9).

12.1.1. What Is a Document? #

A document is the unit of searching in a full text search system; for example, a magazine article or email message. The text search engine must be able to parse documents and store associations of lexemes (key words) with their parent document. Later, these associations are used to search for documents that contain query words.

For searches within PostgreSQL , a document is normally a textual field within a row of a database table, or possibly a combination (concatenation) of such fields, perhaps stored in several tables or obtained dynamically. In other words, a document can be constructed from different parts for indexing and it might not be stored anywhere as a whole. For example:

```
SELECT title || ' ' ||  author || ' ' ||  abstract || ' ' || body AS document
FROM messages
WHERE mid = 12;

SELECT m.title || ' ' || m.author || ' ' || m.abstract || ' ' || d.body AS document
FROM messages m, docs d
WHERE m.mid = d.did AND m.mid = 12;
```

Note

Actually, in these example queries, coalesce should be used to prevent a single NULL attribute from causing a NULL result for the whole document.

Another possibility is to store the documents as simple text files in the file system. In this case, the database can be used to store the full text index and to execute searches, and some unique identifier can be used to retrieve the document from the file system. However, retrieving files from outside the database requires superuser permissions or special function support, so this is usually less convenient than keeping all the data inside PostgreSQL . Also, keeping everything inside the database allows easy access to document metadata to assist in indexing and display.

For text search purposes, each document must be reduced to the preprocessed tsvector format. Searching and ranking are performed entirely on the tsvector representation of a document — the original text need only be retrieved when the document has been selected for display to a user. We therefore often speak of the tsvector as being the document, but of course it is only a compact representation of the full document.

12.1.2. Basic Text Matching #

Full text searching in PostgreSQL is based on the match operator @@ , which returns true if a tsvector (document) matches a tsquery (query). It doesn't matter which data type is written first:

```
SELECT 'a fat cat sat on a mat and ate a fat rat'::tsvector @@ 'cat & rat'::tsquery;
 ?column?
----------
 t

SELECT 'fat & cow'::tsquery @@ 'a fat cat sat on a mat and ate a fat rat'::tsvector;
 ?column?
----------
 f
```

As the above example suggests, a tsquery is not just raw text, any more than a tsvector is. A tsquery contains search terms, which must be already-normalized lexemes, and may combine multiple terms using AND, OR, NOT, and FOLLOWED BY operators. (For syntax details see Section 8.11.2 .) There are functions to_tsquery , plainto_tsquery , and phraseto_tsquery that are helpful in converting user-written text into a proper tsquery , primarily by normalizing words appearing in the text. Similarly, to_tsvector is used to parse and normalize a document string. So in practice a text search match would look more like this:

```
SELECT to_tsvector('fat cats ate fat rats') @@ to_tsquery('fat & rat');
 ?column?
----------
 t
```

Observe that this match would not succeed if written as

```
SELECT 'fat cats ate fat rats'::tsvector @@ to_tsquery('fat & rat');
 ?column?
----------
 f
```

since here no normalization of the word rats will occur. The elements of a tsvector are lexemes, which are assumed already normalized, so rats does not match rat .

The @@ operator also supports text input, allowing explicit conversion of a text string to tsvector or tsquery to be skipped in simple cases. The variants available are:

```
tsvector @@ tsquery
tsquery  @@ tsvector
text @@ tsquery
text @@ text
```

The first two of these we saw already. The form text @@ tsquery is equivalent to to_tsvector(x) @@ y . The form text @@ text is equivalent to to_tsvector(x) @@ plainto_tsquery(y) .

Within a tsquery , the & (AND) operator specifies that both its arguments must appear in the document to have a match. Similarly, the | (OR) operator specifies that at least one of its arguments must appear, while the ! (NOT) operator specifies that its argument must not appear in order to have a match. For example, the query fat & ! rat matches documents that contain fat but not rat .

Searching for phrases is possible with the help of the <-> (FOLLOWED BY) tsquery operator, which matches only if its arguments have matches that are adjacent and in the given order. For example:

```
SELECT to_tsvector('fatal error') @@ to_tsquery('fatal <-> error');
 ?column?
----------
 t

SELECT to_tsvector('error is not fatal') @@ to_tsquery('fatal <-> error');
 ?column?
----------
 f
```

There is a more general version of the FOLLOWED BY operator having the form < N > , where N is an integer standing for the difference between the positions of the matching lexemes. <1> is the same as <-> , while <2> allows exactly one other lexeme to appear between the matches, and so on. The phraseto_tsquery function makes use of this operator to construct a tsquery that can match a multi-word phrase when some of the words are stop words. For example:

```
SELECT phraseto_tsquery('cats ate rats');
       phraseto_tsquery
-------------------------------
 'cat' <-> 'ate' <-> 'rat'

SELECT phraseto_tsquery('the cats ate the rats');
       phraseto_tsquery
-------------------------------
 'cat' <-> 'ate' <2> 'rat'
```

A special case that's sometimes useful is that <0> can be used to require that two patterns match the same word.

Parentheses can be used to control nesting of the tsquery operators. Without parentheses, | binds least tightly, then & , then <-> , and ! most tightly.

It's worth noticing that the AND/OR/NOT operators mean something subtly different when they are within the arguments of a FOLLOWED BY operator than when they are not, because within FOLLOWED BY the exact position of the match is significant. For example, normally !x matches only documents that do not contain x anywhere. But !x <-> y matches y if it is not immediately after an x ; an occurrence of x elsewhere in the document does not prevent a match. Another example is that x & y normally only requires that x and y both appear somewhere in the document, but (x & y) <-> z requires x and y to match at the same place, immediately before a z . Thus this query behaves differently from x <-> z & y <-> z , which will match a document containing two separate sequences x z and y z . (This specific query is useless as written, since x and y could not match at the same place; but with more complex situations such as prefix-match patterns, a query of this form could be useful.)

12.1.3. Configurations #

The above are all simple text search examples. As mentioned before, full text search functionality includes the ability to do many more things: skip indexing certain words (stop words), process synonyms, and use sophisticated parsing, e.g., parse based on more than just white space. This functionality is controlled by text search configurations . PostgreSQL comes with predefined configurations for many languages, and you can easily create your own configurations. (psql 's \dF command shows all available configurations.)

During installation an appropriate configuration is selected and default_text_search_config is set accordingly in postgresql.conf . If you are using the same text search configuration for the entire cluster you can use the value in postgresql.conf . To use different configurations throughout the cluster but the same configuration within any one database, use ALTER DATABASE ... SET . Otherwise, you can set default_text_search_config in each session.

Each text search function that depends on a configuration has an optional regconfig argument, so that the configuration to use can be specified explicitly. default_text_search_config is used only when this argument is omitted.

To make it easier to build custom text search configurations, a configuration is built up from simpler database objects. PostgreSQL 's text search facility provides four types of configuration-related database objects:

• Text search parsers break documents into tokens and classify each token (for example, as words or numbers).

• Text search dictionaries convert tokens to normalized form and reject stop words.

• Text search templates provide the functions underlying dictionaries. (A dictionary simply specifies a template and a set of parameters for the template.)

• Text search configurations select a parser and a set of dictionaries to use to normalize the tokens produced by the parser.

Text search parsers and templates are built from low-level C functions; therefore it requires C programming ability to develop new ones, and superuser privileges to install one into a database. (There are examples of add-on parsers and templates in the contrib/ area of the PostgreSQL distribution.) Since dictionaries and configurations just parameterize and connect together some underlying parsers and templates, no special privilege is needed to create a new dictionary or configuration. Examples of creating custom dictionaries and configurations appear later in this chapter.

12.3. Controlling Text Search

To implement full text searching there must be a function to create a tsvector from a document and a tsquery from a user query. Also, we need to return results in a useful order, so we need a function that compares documents with respect to their relevance to the query. It's also important to be able to display the results nicely. PostgreSQL provides support for all of these functions.

12.3.1. Parsing Documents #

PostgreSQL provides the function to_tsvector for converting a document to the tsvector data type.

```
to_tsvector([ config regconfig, ] document text) returns tsvector
```

to_tsvector parses a textual document into tokens, reduces the tokens to lexemes, and returns a tsvector which lists the lexemes together with their positions in the document. The document is processed according to the specified or default text search configuration. Here is a simple example:

```
SELECT to_tsvector('english', 'a fat  cat sat on a mat - it ate a fat rats');
                  to_tsvector
-----------------------------------------------------
 'ate':9 'cat':3 'fat':2,11 'mat':7 'rat':12 'sat':4
```

In the example above we see that the resulting tsvector does not contain the words a , on , or it , the word rats became rat , and the punctuation sign - was ignored.

The to_tsvector function internally calls a parser which breaks the document text into tokens and assigns a type to each token. For each token, a list of dictionaries (Section 12.6) is consulted, where the list can vary depending on the token type. The first dictionary that recognizes the token emits one or more normalized lexemes to represent the token. For example, rats became rat because one of the dictionaries recognized that the word rats is a plural form of rat . Some words are recognized as stop words (Section 12.6.1), which causes them to be ignored since they occur too frequently to be useful in searching. In our example these are a , on , and it . If no dictionary in the list recognizes the token then it is also ignored. In this example that happened to the punctuation sign - because there are in fact no dictionaries assigned for its token type (Space symbols), meaning space tokens will never be indexed. The choices of parser, dictionaries and which types of tokens to index are determined by the selected text search configuration (Section 12.7). It is possible to have many different configurations in

the same database, and predefined configurations are available for various languages. In our example we used the default configuration english for the English language.

The function setweight can be used to label the entries of a tsvector with a given weight , where a weight is one of the letters A , B , C , or D . This is typically used to mark entries coming from different parts of a document, such as title versus body. Later, this information can be used for ranking of search results.

Because to_tsvector (NULL) will return NULL , it is recommended to use coalesce whenever a field might be null. Here is the recommended method for creating a tsvector from a structured document:

```
UPDATE tt SET ti =
    setweight(to_tsvector(coalesce(title,'')), 'A')    ||
    setweight(to_tsvector(coalesce(keyword,'')), 'B')  ||
    setweight(to_tsvector(coalesce(abstract,'')), 'C') ||
    setweight(to_tsvector(coalesce(body,'')), 'D');
```

Here we have used setweight to label the source of each lexeme in the finished tsvector , and then merged the labeled tsvector values using the tsvector concatenation operator || . (Section 12.4.1 gives details about these operations.)

12.3.2. Parsing Queries #

PostgreSQL provides the functions to_tsquery , plainto_tsquery , phraseto_tsquery and websearch_to_tsquery for converting a query to the tsquery data type. to_tsquery offers access to more features than either plainto_tsquery or phraseto_tsquery , but it is less forgiving about its input. websearch_to_tsquery is a simplified version of to_tsquery with an alternative syntax, similar to the one used by web search engines.

```
to_tsquery([ config regconfig, ] querytext text) returns tsquery
```

to_tsquery creates a tsquery value from querytext , which must consist of single tokens separated by the tsquery operators & (AND), | (OR), ! (NOT), and <-> (FOLLOWED BY), possibly grouped using parentheses. In other words, the input to to_tsquery must already follow the general rules for tsquery input, as described in Section 8.11.2 . The difference is that while basic tsquery input takes the tokens at face value, to_tsquery normalizes each token into a lexeme using the specified or default configuration, and discards any tokens that are stop words according to the configuration. For example:

```
SELECT to_tsquery('english', 'The & Fat & Rats');
  to_tsquery
---------------
 'fat' & 'rat'
```

As in basic tsquery input, weight(s) can be attached to each lexeme to restrict it to match only tsvector lexemes of those weight(s). For example:

```
SELECT to_tsquery('english', 'Fat | Rats:AB');
    to_tsquery
------------------
 'fat' | 'rat':AB
```

Also, * can be attached to a lexeme to specify prefix matching:

```
SELECT to_tsquery('supern:*A & star:A*B');
        to_tsquery
--------------------------
 'supern':*A & 'star':*AB
```

Such a lexeme will match any word in a tsvector that begins with the given string.

to_tsquery can also accept single-quoted phrases. This is primarily useful when the configuration includes a thesaurus dictionary that may trigger on such phrases. In the example below, a thesaurus contains the rule supernovae stars : sn :

```
SELECT to_tsquery('''supernovae stars'' & !crab');
  to_tsquery
---------------
 'sn' & !'crab'
```

Without quotes, to_tsquery will generate a syntax error for tokens that are not separated by an AND, OR, or FOLLOWED BY operator.

```
plainto_tsquery([ config regconfig, ] querytext text) returns tsquery
```

plainto_tsquery transforms the unformatted text querytext to a tsquery value. The text is parsed and normalized much as for to_tsvector , then the & (AND) tsquery operator is inserted between surviving words.

Example:

```
SELECT plainto_tsquery('english', 'The Fat Rats');
 plainto_tsquery
-----------------
 'fat' & 'rat'
```

Note that plainto_tsquery will not recognize tsquery operators, weight labels, or prefix-match labels in its input:

```
SELECT plainto_tsquery('english', 'The Fat & Rats:C');
   plainto_tsquery
---------------------
 'fat' & 'rat' & 'c'
```

Here, all the input punctuation was discarded.

```
phraseto_tsquery([ config regconfig, ] querytext text) returns tsquery
```

phraseto_tsquery behaves much like plainto_tsquery , except that it inserts the <-> (FOLLOWED BY) operator between surviving words instead of the & (AND) operator. Also, stop words are not simply discarded, but are accounted for by inserting < N > operators rather than <-> operators. This function is useful when searching for exact lexeme sequences, since the FOLLOWED BY operators check lexeme order not just the presence of all the lexemes.

Example:

```
SELECT phraseto_tsquery('english', 'The Fat Rats');
 phraseto_tsquery
------------------
 'fat' <-> 'rat'
```

Like plainto_tsquery , the phraseto_tsquery function will not recognize tsquery operators, weight labels, or prefix-match labels in its input:

```
SELECT phraseto_tsquery('english', 'The Fat & Rats:C');
      phraseto_tsquery
-----------------------------
 'fat' <-> 'rat' <-> 'c'
```

```
websearch_to_tsquery([ config regconfig, ] querytext text) returns tsquery
```

websearch_to_tsquery creates a tsquery value from querytext using an alternative syntax in which simple unformatted text is a valid query. Unlike plainto_tsquery and phraseto_tsquery , it also recognizes certain operators. Moreover, this function will never raise syntax errors, which makes it possible to use raw user-supplied input for search. The following syntax is supported:

• unquoted text : text not inside quote marks will be converted to terms separated by & operators, as if processed by plainto_tsquery .

• "quoted text" : text inside quote marks will be converted to terms separated by <-> operators, as if processed by phraseto_tsquery .

• OR : the word “ or ” will be converted to the | operator.

• - : a dash will be converted to the ! operator.

Other punctuation is ignored. So like plainto_tsquery and phraseto_tsquery , the websearch_to_tsquery function will not recognize tsquery operators, weight labels, or prefix-match labels in its input.

Examples:

```
SELECT websearch_to_tsquery('english', 'The fat rats');
 websearch_to_tsquery
----------------------
 'fat' & 'rat'
(1 row)

SELECT websearch_to_tsquery('english', '"supernovae stars" -crab');
       websearch_to_tsquery
---------------------------------
 'supernova' <-> 'star' & !'crab'
(1 row)

SELECT websearch_to_tsquery('english', '"sad cat" or "fat rat"');
       websearch_to_tsquery
-----------------------------------
 'sad' <-> 'cat' | 'fat' <-> 'rat'
(1 row)

SELECT websearch_to_tsquery('english', 'signal -"segmentation fault"');
         websearch_to_tsquery
---------------------------------------
 'signal' & !( 'segment' <-> 'fault' )
(1 row)

SELECT websearch_to_tsquery('english', '""" )( dummy \\ query <->');
 websearch_to_tsquery
----------------------
 'dummi' & 'queri'
(1 row)
```

12.3.3. Ranking Search Results #

Ranking attempts to measure how relevant documents are to a particular query, so that when there are many matches the most relevant ones can be shown first. PostgreSQL provides two predefined ranking functions, which take into account lexical, proximity, and structural information; that is, they consider how often the query terms appear in the document, how close together the terms are in the document, and how important is the part of the document where they occur. However, the concept of relevancy is vague and very application-specific. Different applications might require additional information for ranking, e.g., document modification time. The built-in ranking functions are only examples. You can write your own ranking functions and/or combine their results with additional factors to fit your specific needs.

The two ranking functions currently available are:

• ts_rank([weights float4[] ,] vector tsvector , query tsquery [, normalization integer]) returns float4

Ranks vectors based on the frequency of their matching lexemes.

• ts_rank_cd([weights float4[] ,] vector tsvector , query tsquery [, normalization integer]) returns float4

This function computes the cover density ranking for the given document vector and query, as described in Clarke, Cormack, and Tudhope's "Relevance Ranking for One to Three Term Queries" in the journal "Information Processing and Management", 1999. Cover density is similar to ts_rank ranking except that the proximity of matching lexemes to each other is taken into consideration.

This function requires lexeme positional information to perform its calculation. Therefore, it ignores any " stripped " lexemes in the tsvector . If there are no unstripped lexemes in the input, the result will be zero. (See Section 12.4.1 for more information about the strip function and positional information in tsvector s.)

For both these functions, the optional weights argument offers the ability to weigh word instances more or less heavily depending on how they are labeled. The weight arrays specify how heavily to weigh each category of word, in the order:

```
{D-weight, C-weight, B-weight, A-weight}
```

If no weights are provided, then these defaults are used:

```
{0.1, 0.2, 0.4, 1.0}
```

Typically weights are used to mark words from special areas of the document, like the title or an initial abstract, so they can be treated with more or less importance than words in the document body.

Since a longer document has a greater chance of containing a query term it is reasonable to take into account document size, e.g., a hundred-word document with five instances of a search word is probably more relevant than a thousand-word document with five instances. Both ranking functions take an integer normalization option that specifies whether and how a document's length should impact its rank. The integer option controls several behaviors, so it is a bit mask: you can specify one or more behaviors using | (for example, 2|4).

• 0 (the default) ignores the document length

• 1 divides the rank by 1 + the logarithm of the document length

• 2 divides the rank by the document length

• 4 divides the rank by the mean harmonic distance between extents (this is implemented only by ts_rank_cd)

• 8 divides the rank by the number of unique words in document

• 16 divides the rank by 1 + the logarithm of the number of unique words in document

• 32 divides the rank by itself + 1

If more than one flag bit is specified, the transformations are applied in the order listed.

It is important to note that the ranking functions do not use any global information, so it is impossible to produce a fair normalization to 1% or 100% as sometimes desired. Normalization option 32 (rank/(rank+1)) can be applied to scale all ranks into the range zero to one, but of course this is just a cosmetic change; it will not affect the ordering of the search results.

Here is an example that selects only the ten highest-ranked matches:

```
SELECT title, ts_rank_cd(textsearch, query) AS rank
FROM apod, to_tsquery('neutrino|(dark & matter)') query
WHERE query @@ textsearch
ORDER BY rank DESC
LIMIT 10;
                     title                     |   rank
-----------------------------------------------+----------
 Neutrinos in the Sun                          |      3.1
 The Sudbury Neutrino Detector                 |      2.4
 A MACHO View of Galactic Dark Matter          |  2.01317
 Hot Gas and Dark Matter                       |  1.91171
 The Virgo Cluster: Hot Plasma and Dark Matter |  1.90953
 Rafting for Solar Neutrinos                   |      1.9
 NGC 4650A: Strange Galaxy and Dark Matter     |  1.85774
 Hot Gas and Dark Matter                       |   1.6123
 Ice Fishing for Cosmic Neutrinos              |      1.6
 Weak Lensing Distorts the Universe            | 0.818218
```

This is the same example using normalized ranking:

```
SELECT title, ts_rank_cd(textsearch, query, 32 /* rank/(rank+1) */ ) AS rank
FROM apod, to_tsquery('neutrino|(dark & matter)') query
WHERE  query @@ textsearch
ORDER BY rank DESC
LIMIT 10;
                     title                     |        rank
-----------------------------------------------+-------------------
 Neutrinos in the Sun                          | 0.756097569485493
 The Sudbury Neutrino Detector                 | 0.705882361190954
 A MACHO View of Galactic Dark Matter          | 0.668123210574724
 Hot Gas and Dark Matter                       |  0.65655958650282
 The Virgo Cluster: Hot Plasma and Dark Matter | 0.656301290640973
 Rafting for Solar Neutrinos                   | 0.655172410958162
 NGC 4650A: Strange Galaxy and Dark Matter     | 0.650072921219637
 Hot Gas and Dark Matter                       | 0.617195790024749
 Ice Fishing for Cosmic Neutrinos              | 0.615384618911517
 Weak Lensing Distorts the Universe            | 0.450010798361481
```

Ranking can be expensive since it requires consulting the tsvector of each matching document, which can be I/O bound and therefore slow. Unfortunately, it is almost impossible to avoid since practical queries often result in large numbers of matches.

12.3.4. Highlighting Results #

To present search results it is ideal to show a part of each document and how it is related to the query. Usually, search engines show fragments of the document with marked search terms. PostgreSQL provides a function ts_headline that implements this functionality.

```
ts_headline([ config regconfig, ] document text, query tsquery [, options text ]) returns
text
```

ts_headline accepts a document along with a query, and returns an excerpt from the document in which terms from the query are highlighted. Specifically, the function will use the query to select relevant text fragments, and then highlight all words that appear in the query, even if those word positions do not match the query's restrictions. The configuration to be used to parse the document can be specified by config ; if config is omitted, the default_text_search_config configuration is used.

If an options string is specified it must consist of a comma-separated list of one or more option = value pairs. The available options are:

• MaxWords , MinWords (integers): these numbers determine the longest and shortest headlines to output. The default values are 35 and 15.

• ShortWord (integer): words of this length or less will be dropped at the start and end of a headline, unless they are query terms. The default value of three eliminates common English articles.

• HighlightAll (boolean): if true the whole document will be used as the headline, ignoring the preceding three parameters. The default is false .

• MaxFragments (integer): maximum number of text fragments to display. The default value of zero selects a non-fragment-based headline generation method. A value greater than zero selects fragment-based headline generation (see below).

• StartSel , StopSel (strings): the strings with which to delimit query words appearing in the document, to distinguish them from other excerpted words. The default values are “ ” and “ ” , which can be suitable for HTML output (but see the warning below).

• FragmentDelimiter (string): When more than one fragment is displayed, the fragments will be separated by this string. The default is “ ... ” .

Warning: Cross-site scripting (XSS) safety

The output from ts_headline is not guaranteed to be safe for direct inclusion in web pages. When HighlightAll is false (the default), some simple XML tags are removed from the document, but this is not guaranteed to remove all HTML markup. Therefore, this does not provide an effective defense against attacks such as cross-site scripting (XSS) attacks, when working with untrusted input. To guard against such attacks, all HTML markup should be removed from the input document, or an HTML sanitizer should be used on the output.

These option names are recognized case-insensitively. You must double-quote string values if they contain spaces or commas.

In non-fragment-based headline generation, ts_headline locates matches for the given query and chooses a single one to display, preferring matches that have more query words within the allowed headline length. In fragment-based headline generation, ts_headline locates the query matches and splits each match into “ fragments ” of no more than MaxWords words each, preferring fragments with more query words, and when possible “ stretching ” fragments to include surrounding words. The fragment-based mode is thus more useful when the query matches span large sections of the document, or when it's desirable to display multiple matches. In either mode, if no query matches can be identified, then a single fragment of the first MinWords words in the document will be displayed.

For example:

```
SELECT ts_headline('english',
  'The most common type of search
is to find all documents containing given query terms
and return them in order of their similarity to the
query.',
  to_tsquery('english', 'query & similarity'));
                        ts_headline
------------------------------------------------------------
 containing given <b>query</b> terms                       +
 and return them in order of their <b>similarity</b> to the+
 <b>query</b>.

SELECT ts_headline('english',
  'Search terms may occur
many times in a document,
requiring ranking of the search matches to decide which
occurrences to display in the result.',
  to_tsquery('english', 'search & term'),
  'MaxFragments=10, MaxWords=7, MinWords=3, StartSel=<<, StopSel=>>');
```

```
                          ts_headline
------------------------------------------------------------
 <<Search>> <<terms>> may occur                            +
 many times ... ranking of the <<search>> matches to decide
```

ts_headline uses the original document, not a tsvector summary, so it can be slow and should be used with care.

12.9. Preferred Index Types for Text Search

There are two kinds of indexes that can be used to speed up full text searches: GIN and GiST . Note that indexes are not mandatory for full text searching, but in cases where a column is searched on a regular basis, an index is usually desirable.

To create such an index, do one of:

• CREATE INDEX name ON table USING GIN (column);

Creates a GIN (Generalized Inverted Index)-based index. The column must be of tsvector type.

• CREATE INDEX name ON table USING GIST (column [{ DEFAULT | tsvector_ops } (siglen = number)]);

Creates a GiST (Generalized Search Tree)-based index. The column can be of tsvector or tsquery type. Optional integer parameter siglen determines signature length in bytes (see below for details).

GIN indexes are the preferred text search index type. As inverted indexes, they contain an index entry for each word (lexeme), with a compressed list of matching locations. Multi-word searches can find the first match, then use the index to remove rows that are lacking additional words. GIN indexes store only the words (lexemes) of tsvector values, and not their weight labels. Thus a table row recheck is needed when using a query that involves weights.

A GiST index is lossy , meaning that the index might produce false matches, and it is necessary to check the actual table row to eliminate such false matches. (PostgreSQL does this automatically when needed.) GiST indexes are lossy because each document is represented in the index by a fixed-length signature. The signature length in bytes is determined by the value of the optional integer parameter siglen . The default signature length (when siglen is not specified) is 124 bytes, the maximum signature length is 2024 bytes. The signature is generated by hashing each word into a single bit in an n-bit string, with all these bits OR-ed together to produce an n-bit document signature. When two words hash to the same bit position there will be a false match. If all words in the query have matches (real or false) then the table row must be retrieved to see if the match is correct. Longer signatures lead to a more precise search (scanning a smaller fraction of the index and fewer heap pages), at the cost of a larger index.

A GiST index can be covering, i.e., use the INCLUDE clause. Included columns can have data types without any GiST operator class. Included attributes will be stored uncompressed.

Lossiness causes performance degradation due to unnecessary fetches of table records that turn out to be false matches. Since random access to table records is slow, this limits the usefulness of GiST indexes. The likelihood of false matches depends on several factors, in particular the number of unique words, so using dictionaries to reduce this number is recommended.

Note that GIN index build time can often be improved by increasing maintenance_work_mem , while GiST index build time is not sensitive to that parameter.

Partitioning of big collections and the proper use of GIN and GiST indexes allows the implementation of very fast searches with online update. Partitioning can be done at the database level using table inheritance, or by distributing documents over servers and collecting external search results, e.g., via Foreign Data access. The latter is possible because ranking functions use only local information.